# TRABALHO PRÁTICO 2 - RELATÓRIO DE TRABALHO

MAC0216 - Técnicas de Programação I (2026)

Instituto de Matemática, Estatística e Computação da USP

---




**João Pedro Feitosa - NUSP 10741569**

**Luís Renato Ferraz - NUPS 13726021**

# 1. INTRODUÇÃO

Durante o desenvolvimento em linguagem C, uma limitação técnica frequente é a falta de ferramentas nativas de alto nível para a manipulação de strings. Tarefas consideradas rotineiras em outras linguagens, como fatiamento, substituição de texto e separação de palavras, exigem em C a manutenção direta de ponteiros e a alocação dinâmica de memória, processos que com frequência resultam em vazamentos ou falhas de segmentação (*segmentation fault*) quando mal gerenciados.

Por esse motivo, optou-se por focar o domínio deste projeto no desenvolvimento da biblioteca *str_plus*. O projeto cumpre o propósito de fornecer uma biblioteca com funções seguras e reutilizáveis para o processamento de textos, replicando facilidades de outras linguagens, como o Python, mas mantendo o rigor do gerenciamento de recursos exigido pelo C.

Este relatório documenta o processo de estruturação e implementação da biblioteca *str_plus*. O documento detalha a arquitetura da solução, as decisões de projeto em torno da segurança de memória, o uso do arquivo *Makefile* para a geração de bibliotecas estáticas e dinâmicas, validação com testes automatizados e controle de versão.

# 2. ARQUITETURA DA SOLUÇÃO

A arquitetura do projeto foi projetada com foco na separação de responsabilidades e no encapsulamento de código. Com o objetivo de manter a organização e isolar os arquivos-fonte dos artefatos da compilação, a árvore de diretórios foi estruturada da seguinte maneira:

- ➔ *Makefile*: Arquivo de configuração posicionado na raiz do projeto, responsável por gerenciar o fluxo de compilação, a geração de bibliotecas e execução de testes
- ➔ *include/*: Armazena os arquivos de cabeçalho (*.h*), responsáveis pela interface pública da biblioteca.
- ➔ *src/*: Armazena os arquivos de implementação (*.c*), que contém a lógica interna dos módulos.
- ➔ *app/*: Diretório reservado para a aplicação cliente, contendo o arquivo *main.c*.
- ➔ *tests/*: Isola os arquivos responsáveis pelos testes automatizados.
- ➔ *build/*, *lib/* e *bin/*: Diretórios de saída criados dinamicamente para armazenar, respectivamente, os arquivos objeto (*.o*), as bibliotecas estática e dinâmica compiladas (*.a* e *.so*), e os binários finais executáveis.

A lógica da biblioteca *str_plus* foi dividida em quatro módulos funcionais independentes, garantindo que cada componente possuísse uma responsabilidade coesa e bem definida:

1. **Módulo de Transformação de Caixa** (*str_case*): Responsável por alterar a caixa dos caracteres da string (maísculas, minúsculas e capitalização)
2. **Módulo de Edição** (*str_edit*): Concentra as operações de limpeza de espaços nas extremidades (*strip*), extração de trechos baseada em índices (*slice*) e substituição de ocorrências (*replace*).
3. **Módulo de Contagem** (*str_count*): Agrupa as funções de quantificação, reunindo função de contagem de palavras, linhas não vazias e ocorrências de subtextos sem modificar a *string* original.
4. **Módulo de Tokenização** (***str_token***): Responsável pelo particionamento de textos em vetores de palavras ou linhas, incluindo o fornecimento da função de liberação de memória associada.

Os arquivos *.h* foram definidos como a interface pública, contendo apenas as assinaturas das funções e as proteções de inclusão, o que serve como um manual de uso. Toda a lógica de programação e o gerenciamento de memória ficaram isolados nos arquivos *.c*. Essa divisão visou permitir que o código interno de um módulo pudesse ser consertado ou otimizado sem forçar o programa que utiliza a biblioteca a sofrer qualquer modificação.

O arquivo *main.c* foi construído para atuar apenas como um usuário externo. Ele consome as funções importando exclusivamente os arquivos *.h* necessários, sem realizar nenhum acesso às estruturas internas ou inclusão de arquivos *.c* diretamente.

## 3. API PÚBLICA

A biblioteca *str_plus* é composta por 15 funções públicas, distribuídas entre os quatro módulos do projeto.

As funções da API foram desenvolvidas de tal forma que nunca alteram os textos originais. Com a exceção do módulo de contagem, que retorna valores numéricos, todas as demais funções alocam memória dinamicamente para a geração de novas cadeias de caracteres, com o objetivo de evitar efeitos colaterais indesejados e mutações nos dados originais. Delegou-se ao usuário da biblioteca a responsabilidade de liberar a memória dos ponteiros, utilizando a função padrão *free()* ou a rotina dedicada *str_free_tokens()* para o caso de matrizes.

As funções disponibilizadas em cada módulo são:

1. **Módulo de Transformação de Caixa** (*str_case.h*)
    - *str_upper(str)*: Retorna uma cópia da *string* convertida para letras maiúsculas.
    - *str_lower(str)*: Retorna uma cópia da *string* convertida para letras minúsculas.

    - *str_capitalize(str)*: Retorna uma cópia da *string* com apenas a primeira letra em maiúscula.
    - *str_title(str)*: Retorna uma cópia da *string* com a primeira letra de cada palavra convertida para maiúscula.
2. **Módulo de Edição** (*str_edit.h*)
    - *str_lstrip(str)*: Remove os espaços em branco à esquerda, retornando a nova *string*.
    - *str_rstrip(str)*: Remove os espaços em branco à direita, retornando a nova *string*.
    - *str_strip(str)*: Remove os espaços em branco de ambas as extremidades da *string*.
    - *str_slice(str, start, end)*: Retorna uma fatia da *string* delimitada pelos índices fornecidos, com suporte a índices negativos para contagem a partir do final.
    - *str_replace(str, old, new)*: Retorna uma cópia do texto substituindo todas as ocorrências de uma *substring* por outra.
3. **Módulo de Contagem** (*str_count.h*)
    - *str_wordcount(str)*: Conta e retorna o número de palavras contidas na *string*.
    - *str_linecount(str)*: Conta e retorna o número de linhas não vazias.
    - *str_count(str, wrd)*: Conta quantas vezes uma *substring* específica ocorre dentro do texto.
4. **Módulo de Tokenização** (*str_token.h*)
    - *str_splitwords(str)*: Divide a *string* e retorna um vetor contendo as palavras isoladas.
    - *str_splitlines(str)*: Divide a *string* e retorna um vetor contendo as linhas separadas.
    - *str_free_tokens(tokens, qtd)*: Percorre os níveis de alocação de uma matriz de *tokens* e libera a memória das *strings* individuais e do ponteiro principal, prevenindo vazamentos de memória.

## 4. SISTEMA DE COMPILAÇÃO E BIBLIOTECAS

O processo de compilação do projeto foi totalmente automatizado com um *Makefile*. Para agilizar os testes durante o desenvolvimento, implementou-se a compilação incremental através da regra *build/%.o: src/%.c*, que recompila apenas os arquivos que foram modificados. Além disso, o projeto gera duas versões da biblioteca. A versão estática (*libstr_plus.a*) foi gerada empacotando os arquivos objeto (*.o*). Já para a versão dinâmica (*libstr_plus.so*), os objetos foram compilados com a flag *-fPIC* e a biblioteca foi gerada usando a opção *-shared* do *gcc*.

Na ligação estática, o código da biblioteca é copiado fisicamente para dentro do executável final (*app_estatico*). O arquivo fica maior, mas é totalmente independente e pode rodar em qualquer máquina. No *Makefile*, isso foi feito passando o arquivo *.a* direto no comando do compilador.

Na ligação dinâmica, o executável (*app_dinamico*) guarda apenas uma referência para a biblioteca, ficando menos pesado. A desvantagem é que ele depende do arquivo *.so* existir no sistema na hora de rodar. Para contornar isso nos testes e fazer o executável achar a biblioteca dentro da própria pasta *lib/* sem precisar instalar no sistema, utilizou-se a flag *-Wl,-rpath,lib* na hora da compilação.

## 5. TESTES E VALIDAÇÃO

Para a realização de testes na biblioteca, optou-se por utilizar a ferramenta *assert* em um arquivo independente de testes (*main_test.c*). Os testes cobriram cenários de uso normal e também avaliaram casos limite, verificando o comportamento das funções ao receberem ponteiros nulos (*NULL*), strings vazias (*""*) ou textos compostos apenas por quebras de linha e espaços.

Como a maioria das funções desenvolvidas aloca memória dinamicamente, os testes foram estruturados com uma dupla verificação: primeiro se checa se o ponteiro retornado não é nulo e, em seguida, se compara o texto final com o resultado esperado usando *strcmp*. Após as validações, a memória alocada de cada teste foi devidamente liberada com *free* ou *str_free_tokens*.

Durante o desenvolvimento, um dos testes que falhou inicialmente e precisou de correção ocorreu na função *str_count*. Ao testar a contagem de ocorrências passando uma string vazia (*""*) como alvo da busca, o programa travou em um loop infinito. Isso aconteceu porque o tamanho do alvo era zero, fazendo o ponteiro de leitura não avançar na memória. A falha foi corrigida adicionando uma trava de segurança no início da implementação, que faz a função retornar zero imediatamente caso o tamanho da string buscada seja nula.

## 6. CONTROLE DE VERSÃO (GIT)

Para a coordenação do trabalho em dupla e gerenciamento do projeto, adotou-se controle de versão Git. A estratégia teve como base o desenvolvimento incremental, em que o repositório foi alimentado de forma modular. O histórico registra o avanço gradativo do projeto, progredindo com a adição e finalização de cada um dos módulos de transformação de caixa, edição, contagem e tokenização.

O fluxo de trabalho foi estruturado em torno da *main*, que serviu como o ambiente estável do código. Com o objetivo de evitar conflitos diretos durante a implementação de funcionalidades ou padronizações, também adotou-se o uso de *branches*. Além de ramificações locais utilizadas para a atualização dos módulos, a colaboração remota demandou a criação de múltiplas *branches* de *patch* de autoria individual para isolar certas contribuições.

A integração do código desenvolvido de forma independente (nas *branches*) ocorreu por meio de sucessivos *merges* direcionados à ramificação *main*, garantindo a sincronização segura dos arquivos da dupla. Para delimitar o encerramento do desenvolvimento, estabeleceu-se que, assim que a infraestrutura do *Makefile* e a automação de testes fossem unificadas na ramificação principal, com o repositório recebendo a *tag* ***v1.0***. Essa marcação atesta o congelamento da versão final do projeto, entregando uma biblioteca totalmente compilável e funcional conforme exigido pelo enunciado.

## 7. DIFICULDADES ENCONTRADAS E DECISÕES DE PROJETO

Inicialmente, lidou-se com o comportamento indefinido de funções de caracteres ao processar textos acentuados, o que gerou dificuldades. Isso gerava falhas de acesso na memória, o que exigiu o casting forçado dos caracteres lidos para contornar o problema de leitura dos *bytes*. Outro ponto de ajuste foi a padronização de tipos de variáveis: viu-se a necessidade de revisar os laços de repetição e os retornos das funções para alinhar o uso de *int* e *size_t*, evitando que a contagem estourasse o limite do tipo em *strings* mais longas.

Na manipulação direta dos ponteiros, a tokenização de linhas apresentou problemas com o avanço do cursor de leitura, que, por vezes, travava ao ler o caractere de quebra de linha. Para corrigir isso, foi introduzido um ponteiro auxiliar responsável por isolar a posição de avanço de forma segura, garantindo que a divisão do texto ocorresse sem que ocorresse o corrupção da string. Já no módulo de edição, a dificuldade foi implementar fatiamento com índices negativos, como se tem disponível no fatiamento de strings na linguagem Python. Como o C não suporta isso nativamente, desenvolveu-se uma etapa matemática para converter os valores negativos em índices absolutos, adicionando travas de teto/piso para impedir o vazamento de memória de fora do vetor.

A alocação dinâmica gerou boa parte das falhas de segmentação durante o código. Na função de substituição de texto, trocar palavras por outras de tamanhos diferentes exigiu percorrer a *string* duas vezes, sendo a primeira apenas para contar as ocorrências e calcular matematicamente o novo tamanho total, e a segunda para garantir o tamanho exato da memória antes de fazer a cópia real.

Na infraestrutura, estruturar o *Makefile* exigiu lidar com a sintaxe das regras de dependência do compilador. A principal dificuldade se deu na manipulação das diversas variáveis e comandos. Um gargalo principal foi fazer a compilação incremental funcionar na prática, ajustando as regras para que o *make* avaliasse a data de modificação dos arquivos isoladamente e não recompilasse o projeto inteiro a cada teste de alteração. A geração da biblioteca dinâmica também demandou pesquisa para as *flags* de compilação, para que as rotinas de memória operassem sem erros em tempo de execução quando integradas ao binário final.

## 8. RESULTADOS OBTIDOS E MELHORIAS FUTURAS

O projeto final atendeu aos requisitos estabelecidos, entregando uma biblioteca em C estruturada em módulos independentes e de uso prático, conforme mostrado na main.c. O isolamento entre interface e implementação preservou a separação de tarefas  e a definição de um contrato para a alocação e liberação de ponteiros impede a ocorrência de vazamentos de memória durante a execução de aplicações-cliente. O Makefile cumpre todos os requisitos funcionais, gerando bibliotecas estáticas e dinâmicas e utilizando compilação incremental.

Para o futuro do projeto, uma melhoria técnica interessante seria a implementação de medição de cobertura de testes. Essa adição permitiria mapear com precisão quais erros não estão sendo alcançados pelos casos de teste atuais. Outra evolução técnica relevante seria a adoção de integração contínua utilizando GitHub Actions, o que automatizaria a execução do *Makefile* e da validação a cada novo *merge* na ramificação principal, impedindo a integração de códig que quebre a compilação.

## 9. REPOSITÓRIO

Segue o link para o repositório do GitHub: https://github.com/joao-feitosa/EP2_MAC0216.